

UI/UX Full Automation Testing with Recording — Design Spec

Goal

Full end-to-end UI/UX automation testing across all Yole platforms (Desktop, Web, Android) with real hardware-level interaction (real clicks, real typing, real content creation), video recording of every test run at three human speed modes (slow, normal, fast), and validation that recordings are non-empty and non-black.

Architecture

Platform Coverage

Platform	Automation Tool	Recording Method	Interaction Method
Desktop (Compose JVM)	java.awt.Robot via new ComposeDesktopAdapter	ffmpeg screen capture	Real mouse clicks, real keyboard input
Web (Wasm PWA)	Playwright via existing PlaywrightCLIAdapter	Playwright recordVideo API	Real browser clicks, real form fills
Android	ADB + Espresso via existing adapters	ADB screenrecord	Real touch events, real keyboard input

Speed Modes

Mode	Click Delay	Typing Speed	Navigation Pause	Scroll Speed
slow	800-1200ms	50ms/char	2000ms	500ms/step
normal	300-500ms	30ms/char	1000ms	200ms/step
fast	50-150ms	10ms/char	300ms	50ms/step

Application Flows to Test

Every test covers the complete user journey:

1. **App Launch** — cold start, splash screen, initial load
2. **File Browser** — navigate directories, scroll, select files
3. **Document Creation** — new file, select format, type real content
4. **Document Editing** — modify text, undo/redo, save
5. **Format Switching** — open same content in different formats (all 17)
6. **Preview** — switch to HTML preview, verify rendering
7. **Settings** — change theme (light/dark/system), toggle options
8. **Navigation** — visit every screen/dialog (Files, Todo, QuickNote, Settings, More)

9. **Cloud Storage Setup** — open connection dialogs for all protocols
10. **Keyboard Shortcuts** — test all registered shortcuts (desktop/web)
11. **Search** — search files, search content within documents
12. **Edge Cases** — empty documents, very large files, unicode content

New Components in Challenges Submodule

1. **ComposeDesktopAdapter** (pkg/userflow/adapter_compose_desktop.go)
 - Launches desktop app via `java -jar`
 - Uses `xdotool` for real mouse/keyboard interaction (Linux)
 - Falls back to `java.awt.Robot` via a helper JVM process
 - Supports window focus, click at coordinates, type text, key combos
2. **FFmpegRecorderAdapter** (pkg/userflow/adapter_ffmpeg_recorder.go)
 - Captures screen region via `ffmpeg -f x11grab` (Linux)
 - Configurable FPS, resolution, codec
 - Start/stop recording per test
 - Validates output: file size > 0, duration > 0
3. **SpeedMode type** (pkg/userflow/speed_mode.go)
 - Enum: `Slow`, `Normal`, `Fast`
 - Timing configuration per mode
 - Helper methods: `Sleep()`, `TypeDelay()`, `ClickDelay()`
4. **RecordingValidator** (pkg/userflow/recording_validator.go)
 - Validates video files: duration > 0, no all-black frames
 - Uses `ffprobe` for metadata extraction
 - Uses `ffmpeg` scene detection for black frame check
 - Generates thumbnail screenshots from key frames
5. **Challenge Banks** (3 new JSON files in `banks/yole/`):
 - `ui-automation-desktop.json` — Desktop Compose flows
 - `ui-automation-web.json` — Web/Playwright flows
 - `ui-automation-android.json` — Android ADB/Espresso flows

Recording Storage

- Output: `recordings/` directory (gitignored)
- Structure: `recordings/{platform}/{speed}/{timestamp}-{flow-name}.mp4`
- Screenshots: `recordings/{platform}/{speed}/screenshots/{flow}-{step}.png`
- All gitignored via `.gitignore` patterns

Documentation Deliverables

- `docs/UI_AUTOMATION_TESTING.md` — comprehensive guide
- `docs/user-guide/automation-testing-manual.md` — step-by-step manual

- [docs/diagrams/ui-automation-architecture.mmd](#) — Mermaid diagram
- [video-course/expert/26-ui-automation-testing/script.md](#) — new episode
- Website architecture page update

Constraints

- All interaction **MUST** be real (no mocked UI, no simulated events)
- Recordings **MUST** be validated (non-zero duration, non-black frames)
- Recordings gitignored (too large for repo)
- No sudo/root required — use rootless podman, user-level tools
- Submodule changes **MUST** stay generic and reusable
- All existing tests **MUST** continue to pass